

Rewarding New Ideas in Reader Comments: Design Notes

A plain-language explanation of what I built, why I built it this way, how I judged success, what the results were, and what still doesn't work.

1. The problem in one paragraph

A news site publishes a short article, and readers post comments on it. Most comments repeat the same few reactions ("this will kill local shops", "great for kids"), and a few say something genuinely new. I wanted a system that looks at each new comment and gives it a **reward between 0 and 1**:

- **High** when the comment adds an idea nobody has raised yet, *and* it's actually about the article;
- **Low** when it repeats what others have already said;
- **Zero** when it's about something else entirely, however original it is.

The last point matters most. A comment about the local football team is "new" in the sense that nobody else mentioned football, but it shouldn't earn anything, because it isn't a response to the article.

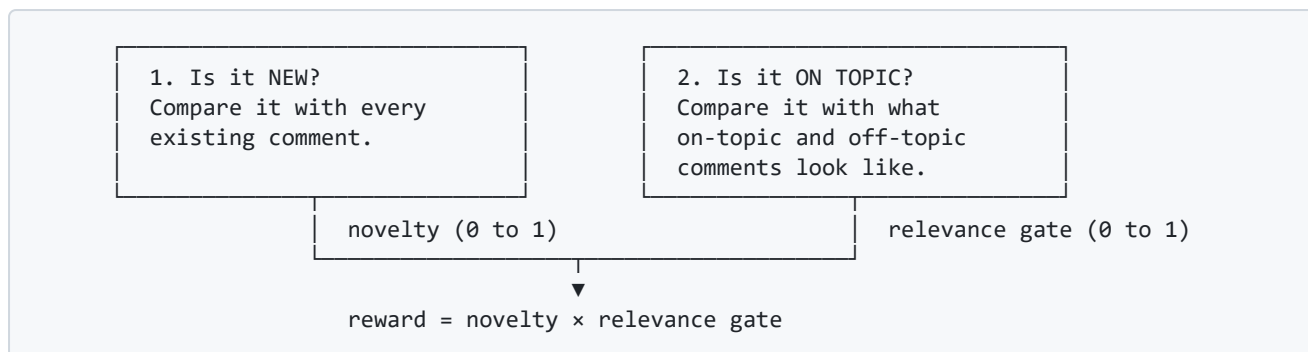
2. What I built, in simple terms

The setting

- **The article:** a 95-word local news story. The Riverton city council votes to tear down a downtown parking garage and build a park, paid for by a business tax and state grants, with a shuttle bus during construction.
- **A comment** has three parts, all written or chosen by the reader:
 - a **headline**,
 - a **body** (the comment itself),
 - a **stance**, picked from *support*, *oppose*, *mixed* or *undecided*.
- **The existing comments:** 50 example comments, written to look like a real comment section. There are many repeats of the obvious takes (parking, families, cost, heat, the shuttle) and only a few one-offs. I made that clustering deliberate: "new" only means something when most comments aren't.

How a comment gets its reward

Every new comment has to answer two questions:



Multiplying the two answers means **both must be yes**. A brilliant idea that's off-topic gets 0, because 0 times anything is 0. So does a perfectly on-topic comment that just repeats others.

How "new" is measured.

- **What the comment is about:** the system turns each comment into a list of numbers that captures its meaning (an "embedding"), so two comments that make the same point sit close together even if they use different words.
- **The words it uses:** it also checks for shared distinctive words, like "stormwater" or "basements".
- **The judgement:** a comment that sits far from every existing comment is new.
- **Relative, not fixed:** "far" isn't a fixed number. The system first measures how far a *typical* existing comment is from the others, and then asks whether the new comment is further out than that. This keeps the reward meaningful even when the comment section changes.

How "on topic" is measured.

- **Learning from examples:** the system learns what this article's topic looks like from two sets of examples. One set is the existing comments on the article. The other is 72 comments from the same town about *other* things: bus routes, the library, water bills, other parks, potholes, school news.
- **The boundary:** it draws the line that best separates the two sets. A new comment on the "other things" side gets a relevance of 0.

Extra checks that can only lower the reward:

- A comment that is a near-copy of an existing one, or of the article itself, gets 0.
- A comment that isn't a real sentence (a list of keywords, one word repeated, a wall of emoji) gets 0.
- A comment that's too vague to be new ("I love this idea!") is scaled down.
- The stance gives a small bonus of up to 10% for a less common stance. It can never turn a copy into a reward.

Cleaning up the text first.

Before any of this, the text is tidied:

- invisible characters and look-alike letters are removed, so a disguised copy is still a copy;
- common typos are fixed and texting shorthand is expanded, so bad spelling neither hides a repeat nor penalises a good idea;
- sentences that aren't in English are set aside rather than guessed at.

Learning as it goes. When a comment is accepted, it joins the pool of existing comments. The next person to make the same point gets a lower reward, which is what "new relative to other submissions"

means. Only comments that are on topic, original enough and actually readable are accepted. That stops spam from changing what "normal" looks like.

3. Why I built it this way

I made each important choice by trying alternatives and measuring the results, not by guessing.

Decision	What I tried first, and why it failed	What I chose
Compare meaning and words	Meaning alone missed rewordings: an already-submitted idea, reworded, still scored 0.60. A bigger language model didn't help (0.81).	Blend meaning (60%) with shared distinctive words (40%). The reworded idea then dropped to about 0.1.
Judge "new" relative to the other comments	Fixed cut-offs behave differently with every language model.	Compare each comment with how spread out the existing comments are.
Check comments point by point	A comment listing every common opinion in one long sentence looked "new", scoring 0.81, because it didn't match any single comment.	Also score each clause separately; the list now scores 0.23.
Learn the topic from examples	Plain similarity to the article, and later "closer to the topic than to generic chatter", both let local comments about <i>other</i> subjects through: bus cuts and library hours scored 0.91. Ready-made AI judges (NLI and reranking models) also misranked comments.	Learn the boundary from on-topic comments versus same-town off-topic comments.
Fix spelling before comparing	Misspelled words looked like new vocabulary, so a typo-filled repeat scored 0.86.	Correct common typos first; the same comment now scores 0.02.
Test on unseen data	My original tests came from the same process that built the system, so they were too easy: they passed while the system failed on realistic comments.	Build an independent test set of 200 comments and report results only on the part never used for tuning.

I followed two engineering principles throughout:

- **Measure before changing.** Every design change and speed-up started with a measurement and ended with a before-and-after number.
- **Keep it modular.** Each check is a separate, pluggable piece. Adding a new one, such as a moderation filter or an AI judge, doesn't require touching the rest.

4. How I defined success

The project brief sets four requirements. I turned each into a measurable bar.

Requirement	What counts as success
Reward truly new, relevant comments	Reward of 0.6 or more
Don't reward repeats	Reward of 0.25 or less for copies and rewordings of existing comments
Don't reward off-topic comments, even very original ones	Reward of 0.01 or less (effectively zero)
Scores on a 0-to-1 scale, relative to the other comments	Every score in [0, 1], and an idea's reward drops once someone has made it

Two kinds of evidence:

1. **Automated tests (189 of them)** that check these bars on hand-picked cases, including about 20 deliberate attempts to cheat the system.
2. **A held-out evaluation** on 200 comments written independently. The authors saw only the article and the example comments, never the scoring code. A second reviewer checked every label without seeing the first one. I split them in two:
 - **dev (70)**: the only part I allowed myself to tune settings on;
 - **test (130)**: set aside and used only to report results, never to tune.

This is the honest measure, because nothing was tuned to fit it.

5. Results

On the hand-picked cases

Kind of comment	Reward
Three genuinely new ideas (flood-proof park design, keeping the garage frame, a heritage walk)	0.84–0.85
Copies and rewordings of existing comments	0.00–0.04
Off-topic comments (sourdough baking, quantum computing, the high-school football team)	0.00
Six local comments about other town issues (bus route, library, water bills, snow plowing, polling place, school hours)	0.00 (previously 0.45–0.97)
Attempts to cheat: stuffing keywords, listing every opinion, repeating a word 40 times, padding with spam, disguised copies	0.00–0.23 (previously up to 0.81)

On the independent test set (130 comments never used for tuning)

What I measured	Before the relevance fix	Now
Off-topic comments given no reward	33%	91%
... on local topics I never trained on	50%	100%
New ideas rewarded	66%	55%
Rewordings of existing comments given no reward	53%	50%
Vague comments given no reward	80%	80%

In plain words:

- **Off-topic comments:** the system now reliably refuses to reward them, including kinds it was never shown. That was the brief's hardest requirement, and the one an independent review found broken before the fix.
- **New ideas:** it rewards about half of them.
- **Rewordings:** it catches about half of the ones that use different words.

Scorecard

Success criterion	Status
Scores on a 0-to-1 scale	Met
Off-topic comments get nothing	Met on unseen data: 91% overall, 100% for topics not in training
Repeats don't get rewarded	Met for close copies and rewordings that reuse words; partly met for rewordings in completely different words (50%)
New ideas get rewarded	Partly met: 55% on unseen comments. Clear new ideas score about 0.85, but subtler ones often fall short of 0.6.
An idea's reward drops once someone has made it	Met when the rewording shares key words (0.86 → 0.06). Not yet met when the rewording uses entirely different words.

Speed and running cost

- **Speed:**
 - scoring a new comment takes about 15–20 thousandths of a second on an ordinary laptop, and about 1 thousandth if it has been seen before;
 - adding an accepted comment and updating everything takes about 4 thousandths.
 - **Cost:** it runs fully offline with a small free language model, so there's no per-comment fee. A cost study I ran found that even with Google's paid embedding service, 1,000 new comments would cost about one to two US cents.
-

6. Limitations: what still doesn't work well

1. **Rewordings in completely different words** are the biggest weakness. If someone restates an existing point with none of the same words, the system often thinks it's new. The fix I'd make next is a second check that asks directly whether two comments make the same point, using a more careful AI comparison.
 2. **The strict topic check has a cost:**
 - about one in ten new ideas that touch a nearby subject is now wrongly treated as off-topic (falcons nesting on the garage, for example);
 - a comment that is half off-topic chatter and half a good idea now earns nothing;
 - two short on-topic comments in the example set (a question about the shuttle, and a complaint about the vote) would be blocked if posted today.
 3. **The closest neighbours of the topic still slip through sometimes:** comments about *other* parks, library funding and recycling.
 4. **Everything was checked by machine, not people.** The example comments, the independent test comments and their labels were all produced by AI systems from the same family as the AI coding agent I built the scorer with (see COLLABORATION.md). A proper evaluation would use real comments labelled by people.
 5. **It's built for one article.** The topic check learns from off-topic examples written for this article (I used 72), so a new article needs its own set. With fewer than 10, it falls back to a simpler check that is known to be weaker.
 6. **English only.** Other languages are set aside rather than scored.
 7. **It judges newness, not quality or truth.** A new but rude or false comment is rewarded like any other new comment. A real deployment would put this behind moderation.
 8. **The Google (Gemini) option hasn't been tried** with a real account.
-

7. Glossary

Term	Meaning here
Embedding	A list of numbers that captures what a piece of text means, so similar meanings get similar numbers.
Novelty	How far a comment is from every existing comment, compared with how far apart the existing comments usually are.
Relevance gate	A number from 0 to 1 that multiplies the reward: 0 if the comment isn't about the article, 1 if it clearly is.
Hard negatives	Off-topic examples that look deceptively similar to on-topic ones (same town, same tone), used to teach the topic check where the line is.
Dev / test split	The evaluation comments divided in two: settings are tuned only on <i>dev</i> , and results are reported on <i>test</i> , so the reported numbers aren't flattered by tuning.
AUC	A score from 0.5 (no better than guessing) to 1.0 (perfect) for how well the system ranks one kind of comment above another.